

Week 3 Study Material

EDA Design Flow Integration: FIR Filter Architecture and Delay-Line Thinking

Item	Details
Course	EDA Design Flow Integration
Week	3
Main Topic	Guided Project - FIR Filter Architecture
Guided Project	Fixed-point FIR filter (Week 3 uses simple integer tracing; bit-width refinement comes later)
Evaluation Project	Small fixed-point matrix multiplication engine
Next Week Target	Sequential FIR RTL design and simulation
Scope	Software-only architecture, tracing, and simulation preparation; no FPGA board execution or ASIC tapeout/silicon validation

1. Why Week 3 Starts the Guided Project

Week 1 introduced the overall EDA design-flow idea. Week 2 introduced small Verilog RTL, testbenches, and waveform reading. Week 3 now begins the guided project: a fixed-point FIR filter.

The goal this week is architecture understanding. Students should understand the delay line, taps, coefficients, multipliers, adders, reset behavior, valid-controlled sample acceptance, and expected outputs before writing the full RTL in Week 4.

Week 3 remains software-only: no FPGA board execution is required, and no ASIC tapeout, fabrication, or silicon validation is involved. Evidence comes from hand tracing, architecture reasoning, and simulation-oriented preparation.

2. Course Scoring Context

Course component	Weight	Why Week 3 matters
Xilinx FPGA evaluation project	45%	The FIR project builds the architecture and evidence habits used before FPGA implementation.
Cell-based IC evaluation project	45%	Clean architecture thinking prepares students for later synthesis, timing, area, and power analysis.
Attendance and participation	10%	Week 3 guided activity, discussion, and worksheet completion support this component.

3. What a FIR Filter Is

A finite impulse response (FIR) filter produces each output sample by combining a finite number of recent input samples. It is common in signal processing, but in this course it is used mainly because it is an excellent digital-hardware example.

An FIR filter is useful for learning EDA flows because it contains storage, arithmetic, timing paths, and design trade-offs, but it can still be explained using small examples.

4. Samples, Coefficients, and Taps

The input samples are written as $x[n]$, $x[n-1]$, $x[n-2]$, and so on. The newest accepted sample is $x[n]$. Older accepted samples are stored in delay registers. Coefficients are fixed values that weight each sample path.

Term	Meaning
$x[n]$	Current input sample
$x[n-1]$	Input sample from one previous cycle
$x[n-2]$	Input sample from two previous cycles
h_0, h_1, h_2	Coefficient values
Tap	One sample path - current or delayed - weighted by one coefficient
d_0, d_1, d_2	Week 3 register outputs: $d_0 = x[n]$, $d_1 = x[n-1]$, $d_2 = x[n-2]$ after an accepted rising edge

Week 3 convention: after a rising clock edge that accepts a sample, d_0 stores $x[n]$, d_1 stores $x[n-1]$, and d_2 stores $x[n-2]$. This convention is used consistently in the slides, handout, and source package.

5. Three-Tap FIR Equation

A small three-tap FIR can be written as:

$$y[n] = h_0 * x[n] + h_1 * x[n-1] + h_2 * x[n-2]$$

This means that the output is not based only on the current sample. It also depends on stored history. In the Week 3 register view, the same equation is $y_out = h_0 * d_0 + h_1 * d_1 + h_2 * d_2$.

6. Hand Computation Example

Use coefficients $h_0 = 1$, $h_1 = 2$, $h_2 = 1$ and input sequence $x = 1, 2, 3, 0, 0$. Assume $d_0 = d_1 = d_2 = 0$ before the first sample is accepted, rst_n has already been released, and $valid_in = 1$ for each listed sample.

n	x_in	$d_0 = x[n]$	$d_1 = x[n-1]$	$d_2 = x[n-2]$	Expected y_out
0	1	1	0	0	1
1	2	2	1	0	4
2	3	3	2	1	8
3	0	0	3	2	8
4	0	0	0	3	3

The early outputs include zeros from the reset history. In this simple Week 3 architecture, these are legitimate zero-padded FIR results: 1, 4, 8, 8, 3. Do not invent a fixed three-cycle fill delay.

7. Delay Line Thinking

A delay line is a chain of registers. At a rising clock edge with $valid_in = 1$, the newest input sample is accepted into d_0 , while the old d_0 and d_1 values move to d_1 and d_2 . If $valid_in = 0$, d_0 , d_1 , and d_2 hold their previous values.

When $valid_in = 1$ at a rising edge:

```
d0 <= x_in;  
d1 <= d0;  
d2 <= d1;
```

These nonblocking assignments update together on the same rising clock edge. Their right-hand-side values are the old values from before the edge: $d0_new = x_in$, $d1_new = d0_old$, and $d2_new = d1_old$.

8. Reset Behavior and Output Timing

The active-low reset rst_n gives the delay line a known starting state. In the Week 4-style register model, asserting $rst_n = 0$ clears $d0$, $d1$, and $d2$. The Week 3 arithmetic output is combinational from those registers, so zeroed history makes y_out evaluate from zeros.

- Before reset release, $d0 = d1 = d2 = 0$.
- Reset release ($rst_n: 0 \rightarrow 1$) does not itself capture or shift a sample. Normal capture occurs on a later rising clock edge when $valid_in = 1$.
- No extra output register is assumed in Week 3; y_out responds combinatorially to the current $d0/d1/d2$ state after propagation delay.
- Early outputs are zero-padded FIR results from reset history; do not automatically discard them for a fixed number of cycles.

9. Multiply-and-Add Datapath

Each tap creates one explicit product. For the Week 3 register convention: $p0 = h0*d0$, $p1 = h1*d1$, and $p2 = h2*d2$. The three products are combined with two explicit additions: $s0 = p0 + p1$, then $y_out = s0 + p2$.

Datapath part	Role
Multiplier 0	Computes $p0 = h0*d0$
Multiplier 1	Computes $p1 = h1*d1$
Multiplier 2	Computes $p2 = h2*d2$
Adder 1	Computes $s0 = p0 + p1$
Adder 2 / y_out	Computes $y_out = s0 + p2$; no extra output register is assumed in Week 3

10. Sequential and Combinational Views

The FIR architecture contains sequential state, combinational arithmetic, constants, and control. The delay line is sequential because it stores sample history. Multipliers and adders are combinational in the simple Week 3 view because they compute from the currently stored register outputs.

Part	View	Reason
Delay registers $d0/d1/d2$	Sequential storage	They store accepted sample history across clock cycles.
Reset behavior	Sequential control	It initializes or clears the stored register state.
Coefficient $h0/h1/h2$	Constants	They are fixed values used by the datapath in this example.
Multipliers $p0/p1/p2$	Combinational	They compute products from the currently available register values.
Adder network / y_out	Combinational	It combines products without adding storage in the Week 3 model.
$valid_in$ / optional $valid_out$	Control	$valid_in$ controls sample acceptance; $valid_out$ is used only if the final RTL explicitly defines it and its latency.

11. Fixed-Point Intuition and Width Growth

A fixed-point value represents a value with a fixed binary-point position within a fixed-width word. Week 3 uses small integer values so that students can focus on data movement and arithmetic structure before introducing fractional formats.

- For signed 8-bit samples, x is in the range -128 to 127.
- With $h = [1, 2, 1]$, p_0 and p_2 range from -128 to 127, while p_1 ranges from -256 to 254.
- The partial sum s_0 ranges from -384 to 381 and the final y range is -512 to 508, so 10 signed bits are sufficient for this specific coefficient set.
- The supplied generic arithmetic reference is deliberately wider (16-bit products, 17-bit s_0 , 18-bit y_{out}) because it accepts general signed 8-bit coefficients; this does not contradict the 10-bit minimum for the fixed $[1, 2, 1]$ example.

12. Testbench Thinking for FIR

An FIR testbench should not only apply input samples; it should know the expected state and output sequence. Expected values should be calculated from the equation and sample history before trusting a waveform.

Testbench task	Why it matters
Apply reset	Starts $d_0/d_1/d_2$ from a known zero history.
Release reset between capture edges	Prevents reset deassertion from being confused with sample capture.
Drive x_{in} and $valid_{in}$	Creates a controlled accepted/held input sequence.
Observe rising clock edges	Matches the register update event.
Check $d_0/d_1/d_2$ and y_{out}	Confirms both sample history and arithmetic correctness.
Inspect $p_0/p_1/p_2$ and s_0	Helps localize arithmetic errors instead of looking only at y_{out} .
Save logs and waveforms	Provides simulation evidence that can be compared with the hand calculation.

13. Week 3 Source Package Boundary

The accompanying source package supports Week 3 without giving away the Week 4 task. `rtl/fir3_arith.v` is a synthesizable combinational arithmetic reference: d_0 , d_1 , and d_2 are already-stored sample inputs, and the file exposes p_0 , p_1 , p_2 , s_0 , and y_{out} . `tb/tb_fir3_trace.v` uses testbench-only state to reproduce the hand-trace sequence.

The package intentionally does not contain the complete edge-triggered FIR delay-line RTL. Students will write the synthesizable sequential register chain in Week 4. Source-code execution is supporting evidence, not a replacement for the Week 3 hand calculation and explanation.

14. Waveform Signals to Expect

Signal	What to inspect
<code>clk</code>	Rising edges where delay registers may update.
<code>rst_n</code>	When active-low reset is asserted and when it is released.
<code>x_in</code>	Incoming sample value.
<code>valid_in</code>	Whether x_{in} is accepted on a rising edge; when 0, the delay line holds.
<code>d0</code> , <code>d1</code> , <code>d2</code>	Accepted sample history and simultaneous movement through the delay line.

Signal	What to inspect
p0, p1, p2	One product per tap.
s0	Partial sum p0 + p1.
y_out	Combinational FIR result s0 + p2.
valid_out (optional)	Inspect only if the Week 4 RTL explicitly defines this signal and its latency; do not invent a fixed valid delay.

15. Connection to the Two EDA Flows

The same FIR architecture will later be examined through both course flows. These are implementation and report-reading exercises, not board or silicon-validation activities.

Flow	Architecture connection	Later evidence
FPGA flow	Delay registers may map to flip-flops; small arithmetic/control may use LUT/carry logic; multipliers may map to DSP blocks or LUT logic depending on widths, constants, target, and synthesis decisions.	Timing and utilization reports; bitstream file as a software artifact.
Cell-based digital IC flow	Registers and arithmetic are synthesized into standard-cell logic and then implemented.	Timing, area, and power reports; no tapeout or silicon validation.

16. Week 3 Correctness Rules

- d0, d1, and d2 update simultaneously from the old state on an accepted rising edge.
- A new sample is accepted only when valid_in = 1 at a rising edge; when valid_in = 0, the delay line holds.
- Reset release does not itself capture a sample.
- Early outputs are zero-padded FIR results from reset history; do not invent a fixed three-cycle fill delay.
- y_out is combinational in the Week 3 model; no extra output register is assumed.
- Each multiplier has two explicit inputs (sample and coefficient), and all three products must enter the two-adder chain.
- Debug the complete chain rst_n / clk / valid_in -> x_in -> d0/d1/d2 -> p0/p1/p2 -> s0 -> y_out, not only the final output.

17. How Week 3 Supports Week 4

Week 4 will convert the FIR architecture into sequential plain-Verilog RTL and verify it with a testbench. The key Week 3 skill is knowing exactly what the RTL must do before writing it.

Week 3 concept	Week 4 RTL connection
Delay line	Register chain using nonblocking assignments on posedge clk, with active-low reset.
Tap	One current or delayed sample path multiplied by one coefficient.
valid_in	Accept a new sample on a rising edge only when valid_in = 1; otherwise hold state.
Hand calculation	Expected state/output sequence for self-checking testbench comparisons.
Waveform prediction	Signal list for simulation evidence, including internal arithmetic where useful.
Sequential/combinational classification	Clean separation of clocked storage from combinational arithmetic.
Output timing	Combinational y_out from the current stored state unless a later architecture explicitly adds output pipelining.

18. Self-Check Questions

What is a tap in an FIR filter?

Why does a FIR filter need delay registers?

Why are early FIR outputs affected by reset history?

Which parts of a simple FIR are sequential?

Which parts of a simple FIR are combinational?

Why should expected outputs be calculated before simulation?

How does Week 3 prepare for Week 4 sequential FIR RTL?

What happens to d0, d1, and d2 when valid_in = 0 at a rising clock edge?

Does changing rst_n from 0 to 1 itself capture an input sample? Explain.

Why should you not automatically wait three cycles before considering the first FIR output meaningful in this Week 3 model?

Why can the source package use wider arithmetic signals than the minimum width calculated for $h = [1, 2, 1]$?
